

Minimum Pushes to Type Word

Observation

- We can assign letters to keypad positions in any order.
- To minimize pushes, assign the most frequently used letters to the lowest push count.
- In this problem, every character is unique (or effectively we're counting distinct positions), so only the number of characters matters.

Keypad Capacity

1st push → 8 letters

2nd push → next 8 letters

3rd push → next 8 letters

4th push → remaining 2 letters
(since only 26 alphabets)

Greedy Idea

Always fill the cheapest push slots first.

First 8 chars → 1 push each

Next 8 chars → 2 pushes each

Next 8 chars → 3 pushes each

Last 2 chars → 4 pushes each

Pattern Used

```
take = min(remainingCharacters, 8)
```

```
answer += take × currentPushCost
```

```
remainingCharacters -= take
```

Repeat for push costs 1, 2, 3, 4.

Code Implementation

```
class Solution {
public:
    int minimumPushes(string word) {
        int ans=0;
        int n=word.size();

        if(n>0){
            ans+=min(n,8); //eg 5 aur 25 , 5 he to 5 keys me pehle
            //25 hua to , pehle 8 , bach gaye 17
            n-=min(n,8);
        }

        if(n>0){
            ans+=min(2*n,16); //17 me se , 16 taps pehle 8 me for 8
            //bach gaye ab 9 char
            n-=min(n,8);
        }

        if(n>0){
            ans+=min(3*n,24); //max ab 24 aur tap me 8 character aur
            //bach gaye ab 9-8=1 char
            n-=min(n,8);
        }

        if(n>0){
            ans+=min(4*n,32); //just 4 more taps for last char
            n-=min(n,8); //no more char left
        }

        return ans;
    }
};
```